

LAPORAN PRAKTIKUM

(CLOUD COMPUTING)

Dosen Pengampu: Bpk. Jaka Sutresna S.Kom.M.Kom



Disusun oleh:

Nama : Dini Yusriah

NIM : 241011403194

Kelas : 04TPLP008

TEKNIK INFORMATIKA
FAKULTAS ILMU KOMPUTER
UNIVERSITAS PAMULANG

TAHUN 2025/2026
KATA PENGANTAR

Puji dan syukur penulis panjatkan ke hadirat Tuhan Yang Maha Esa atas rahmat dan karunia-Nya sehingga laporan praktikum mata kuliah Cloud Computing ini dapat diselesaikan dengan baik. Laporan ini disusun sebagai bentuk dokumentasi dan pertanggungjawaban atas kegiatan praktikum yang telah dilakukan, meliputi pembelajaran deployment web, penggunaan Virtual Machine dengan Oracle VirtualBox dan Ubuntu Server, serta implementasi container menggunakan Docker dan Docker Compose.

Penulis mengucapkan terima kasih kepada Bapak Jaka Sutresna, S.Kom., M.Kom selaku dosen pengampu yang telah memberikan bimbingan selama proses praktikum. Penulis menyadari bahwa laporan ini masih memiliki kekurangan, sehingga kritik dan saran yang membangun sangat diharapkan untuk perbaikan di masa mendatang.

Bogor, 22 April 2026.

Dini Yusriah

DAFTAR ISI

KATA PENGANTAR	ii
DAFTAR ISI	iii
PERTEMUAN 4.....	1
(DEPLOY WEB MENGGUNAKAN NGINX GITHUB CODESPACES).....	1
LANDASAN TEORI.....	1
PERTEMUAN 5	4
Virtual Box dan Ubuntu Server	4
LANDASAN TEORI.....	4
PERTEMUAN 7.....	10
SETUP DOCKER.....	10
LANDASAN TEORI.....	10
PERTEMUAN 8.....	13
POWER OFF VIRTUAL MACHINE	13
LANDASAN TEORI.....	13
KESIMPULAN	18

PERTEMUAN 4

(DEPLOY WEB MENGGUNAKAN NGINX GITHUB CODESPACES)

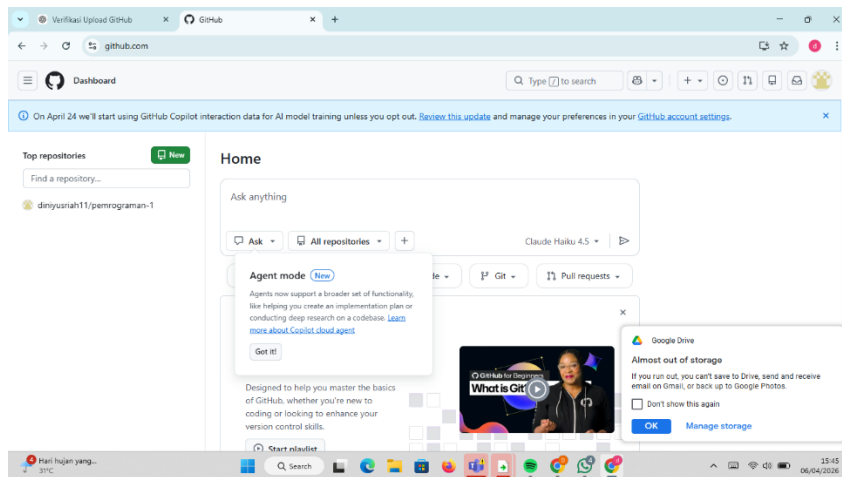
LANDASAN TEORI

Website merupakan kumpulan halaman yang berisi informasi dalam bentuk teks, gambar, maupun multimedia yang dapat diakses melalui internet menggunakan browser. Agar sebuah website dapat diakses oleh pengguna, diperlukan web server yang berfungsi untuk menerima permintaan dari client dan mengirimkan respon berupa halaman web. Salah satu web server yang sering digunakan adalah Nginx, yang dikenal memiliki performa tinggi dan efisien dalam menangani banyak permintaan secara bersamaan.

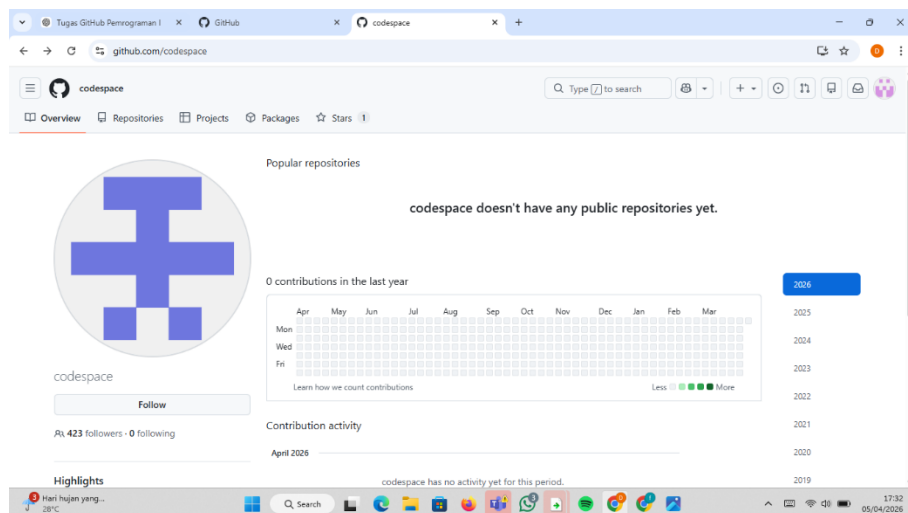
Dalam proses pengembangan modern, pembuatan dan pengujian website tidak selalu dilakukan secara lokal, melainkan dapat menggunakan lingkungan berbasis cloud. Salah satu layanan yang mendukung hal tersebut adalah GitHub Codespaces, yaitu sebuah platform yang menyediakan environment pengembangan langsung dari repository GitHub tanpa perlu instalasi tambahan di komputer pengguna. Dengan layanan ini, pengguna dapat menulis, menjalankan, dan menguji aplikasi web secara langsung melalui browser.

Proses menjalankan website agar dapat diakses disebut sebagai deployment. Dalam praktikum ini, deployment dilakukan dengan cara menjalankan web server di dalam GitHub Codespaces, kemudian membuka akses ke aplikasi melalui fitur port forwarding. Fitur ini memungkinkan aplikasi yang berjalan di dalam lingkungan cloud dapat diakses dari luar seolah-olah seperti website yang dihosting secara online.

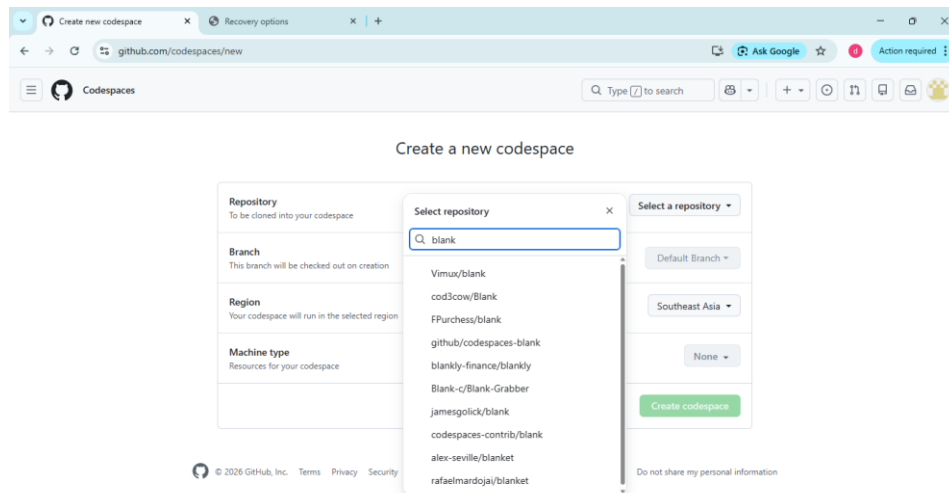
Dengan demikian, penggunaan GitHub Codespaces dan web server seperti Nginx mempermudah proses pengembangan dan pengujian website, karena seluruh proses dapat dilakukan secara praktis tanpa perlu konfigurasi server secara manual di perangkat lokal.



Pada langkah ini saya membuka dan login ke **GitHub**. Setelah berhasil login, muncul halaman **Dashboard** yang merupakan halaman utama akun GitHub untuk mengelola repository dan mengakses fitur lain seperti **GitHub Codespaces**. Halaman ini digunakan sebagai langkah awal sebelum membuat Codespaces untuk menjalankan web server.



Pada langkah ini saya membuka halaman utama akun di **GitHub** setelah berhasil login. Halaman ini menampilkan profil akun, aktivitas kontribusi, serta repository yang dimiliki oleh pengguna. Dari halaman ini pengguna juga dapat mengakses berbagai fitur GitHub seperti repository, project, dan layanan **GitHub Codespaces** untuk menjalankan lingkungan pemrograman berbasis cloud.



Pada tahap ini saya memilih template **github/codespaces-blank** di GitHub Codespaces. Template ini digunakan untuk membuat environment kerja yang masih kosong sehingga dapat digunakan untuk menjalankan perintah instalasi dan membuat file web server menggunakan Nginx.

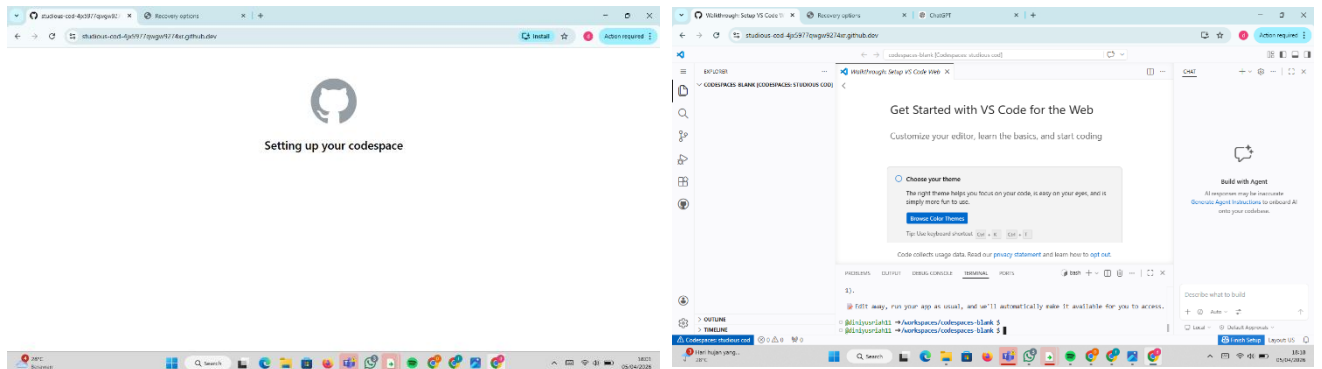
PERTEMUAN 5

Virtual Box dan Ubuntu Server

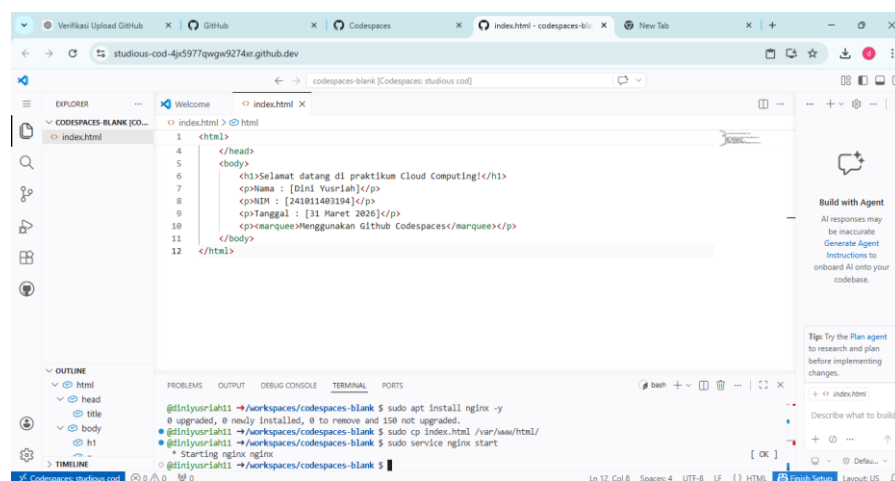
LANDASAN TEORI

Virtualisasi merupakan teknologi yang memungkinkan satu perangkat komputer menjalankan beberapa sistem operasi secara bersamaan dalam bentuk mesin virtual. Salah satu software yang digunakan untuk melakukan virtualisasi adalah Oracle VM VirtualBox, yang memungkinkan pengguna membuat dan menjalankan sistem operasi lain di dalam komputer tanpa mengganggu sistem utama. Dengan VirtualBox, pengguna dapat melakukan simulasi server untuk keperluan pembelajaran maupun pengujian. Salah satu sistem operasi yang umum digunakan dalam lingkungan server adalah Ubuntu Server. Ubuntu Server merupakan sistem operasi berbasis Linux yang dirancang untuk kebutuhan server, seperti hosting website, pengelolaan jaringan, dan layanan lainnya. Sistem ini dikenal stabil, ringan, serta memiliki dukungan komunitas yang luas sehingga banyak digunakan dalam proses pembelajaran maupun implementasi nyata.

Untuk menampilkan halaman website, diperlukan web server yang bertugas melayani permintaan dari client dan mengirimkan halaman web sebagai respon. Dalam praktikum ini digunakan Nginx sebagai web server karena memiliki performa yang baik dan konfigurasi yang relatif sederhana. Nginx mampu menampilkan file website seperti HTML sehingga dapat diakses melalui browser. Website sederhana biasanya dibuat menggunakan HTML (HyperText Markup Language), yaitu bahasa dasar untuk membuat struktur halaman web. Dalam praktikum ini, website yang dibuat hanya berupa tampilan sederhana, misalnya menampilkan nama pengguna, yang disimpan dalam file HTML dan diletakkan pada direktori web server agar dapat diakses. Dengan menggunakan VirtualBox, Ubuntu Server, dan Nginx, proses pembuatan dan pengujian website dapat dilakukan dalam lingkungan virtual tanpa memerlukan server fisik. Hal ini memudahkan pengguna dalam memahami konsep dasar server dan deployment web secara langsung.



Setelah memilih template **Blank Codespace**, sistem akan melakukan proses **setting up your codespace**. Pada tahap ini GitHub menyiapkan lingkungan kerja berbasis **VS Code di browser**. Setelah proses selesai, akan terbuka tampilan **VS Code Web** yang berisi file explorer, terminal, dan editor kode yang digunakan untuk membuat dan menjalankan program.



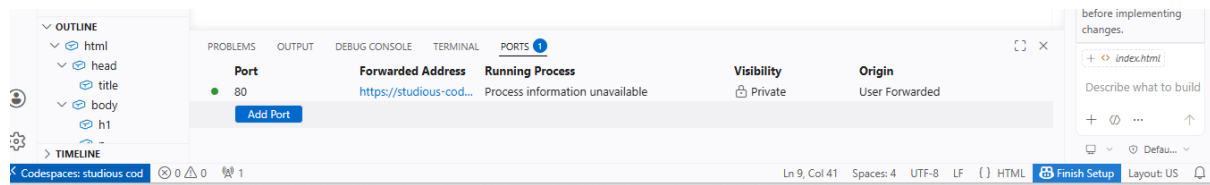
Masuk ke GitHub dan membuka GitHub Codespaces untuk membuat environment cloud.

Menginstall web server Nginx melalui terminal dengan perintah:

`sudo apt install nginx -y` untuk menginstall web server **Nginx**.

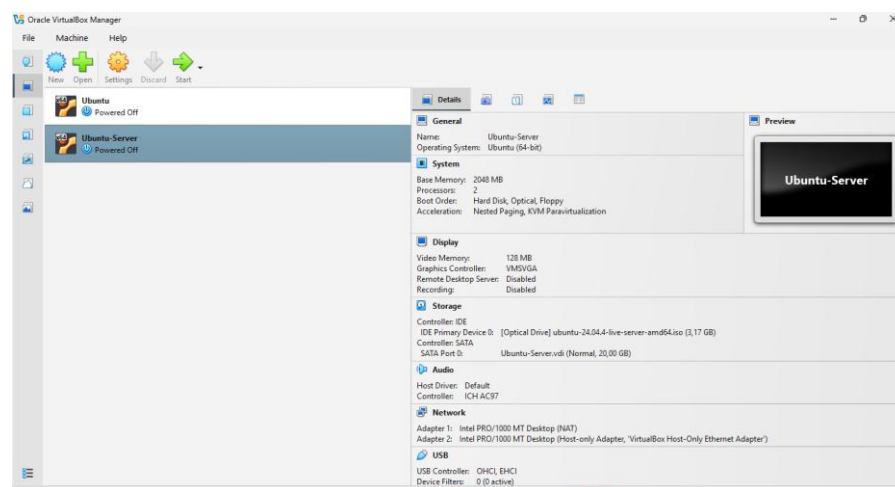
`Sudo cp index.html /var/www/html/` untuk menyalin file **index.html** ke folder web server.

`sudo service nginx start` untuk menjalankan layanan **Nginx** agar website dapat diakses melalui browser.

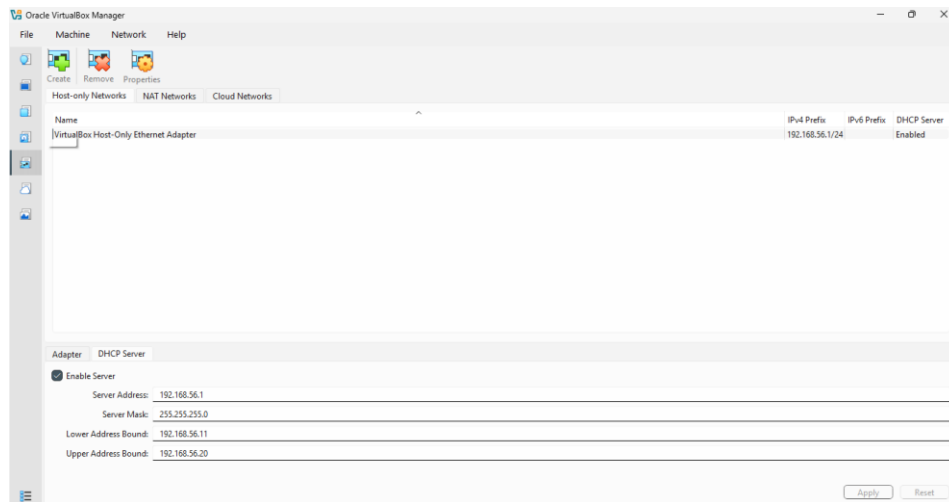


Port 80 adalah jalur komunikasi standar yang digunakan untuk mengakses website melalui web server. Pada saat Nginx dijalankan, server akan aktif di port 80 sehingga website dapat diakses melalui browser.

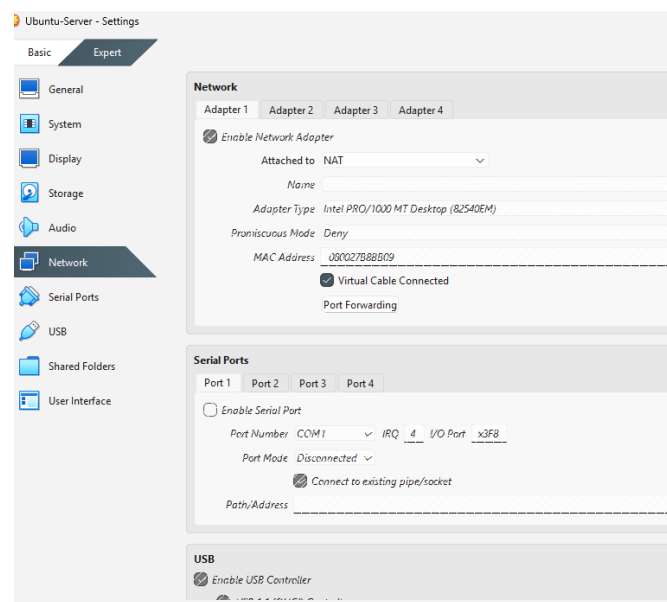
Ketika muncul notifikasi “aplikasi berjalan pada port 80”, pengguna dapat membuka website dengan klik “Open in Browser” untuk melihat hasil dari file HTML yang telah dibuat.



Ini adalah tampilan dari Oracle VM VirtualBox, yaitu aplikasi yang digunakan untuk menjalankan sistem operasi lain secara virtual di dalam satu komputer. Pada gambar terlihat ada dua mesin virtual, yaitu Ubuntu dan Ubuntu-Server, yang keduanya masih dalam keadaan mati (Powered Off). Bagian kanan menampilkan detail dari Ubuntu-Server seperti jumlah RAM, kapasitas penyimpanan, dan file installer yang digunakan. Ubuntu-Server sendiri biasanya dipakai untuk keperluan server, misalnya membuat web sederhana menggunakan Apache atau Nginx, jadi ini bukan langsung websitenya, melainkan lingkungan atau “komputer virtual” tempat kamu nanti menginstal dan menjalankan web server tersebut.

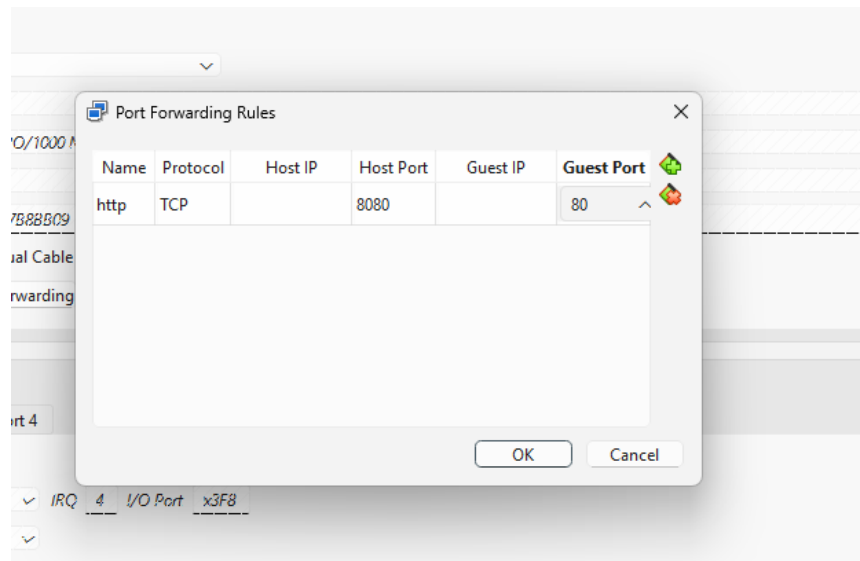


bagian dari Oracle VM VirtualBox, tapi khusus di menu **Host-Only Network**. Di sini kamu sedang melihat pengaturan jaringan virtual yang memungkinkan komputer asli (host) berkomunikasi langsung dengan mesin virtual (misalnya Ubuntu-Server), tanpa perlu koneksi internet luar. Terlihat ada adapter bernama “VirtualBox Host-Only Ethernet Adapter” dengan IP 192.168.56.1, serta DHCP Server yang aktif untuk membagikan IP otomatis ke VM dalam rentang 192.168.56.11–192.168.56.20. Pengaturan ini biasanya dipakai saat membuat server lokal (misalnya web server), supaya kamu bisa mengakses web dari VM lewat komputer kamu sendiri menggunakan IP tersebut

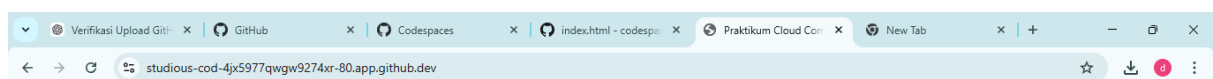


pengaturan **Network** pada VM di Oracle VM VirtualBox. Terlihat Adapter 1 aktif dan menggunakan mode **NAT**, artinya mesin virtual bisa mengakses internet lewat koneksi komputer kamu, tapi tidak langsung terlihat dari jaringan luar. Opsi “Virtual Cable Connected” menandakan jaringan

tersambung. Pengaturan ini cocok untuk kebutuhan dasar seperti update sistem atau install web server, tapi kalau mau akses web dari luar (misalnya dari browser di host), biasanya perlu tambahan setting seperti port forwarding atau ganti ke host-only/bridged.



pengaturan Port Forwarding di Oracle VM VirtualBox. Aturan yang kamu buat artinya: ketika kamu akses localhost:8080 di computer (host), maka akan diteruskan ke **port 80** di dalam VM (Ubuntu-Server). Port 80 biasanya dipakai untuk web server (HTTP), jadi ini memungkinkan kamu membuka web yang ada di VM lewat browser di host tanpa perlu IP khusus. Singkatnya, 8080 (host) → 80 (server web di VM).



Selamat datang di praktikum Cloud Computing!

Nama : [Dini Yusrilah]

NIM : [241011403194]

Tanggal : [31 Maret 2026]

Menggunakan Github Codespaces

Ini adalah tampilan halaman web sederhana yang kamu jalankan lewat GitHub Codespaces. Isinya berupa HTML dasar yang menampilkan judul “Selamat datang di praktikum Cloud Computing”, lalu ada data seperti nama, NIM, dan tanggal. Artinya kamu sudah berhasil menjalankan web statis di environment cloud (Codespaces), jadi ini hasil dari file HTML yang kamu buat dan dijalankan di browser.

PERTEMUAN 7

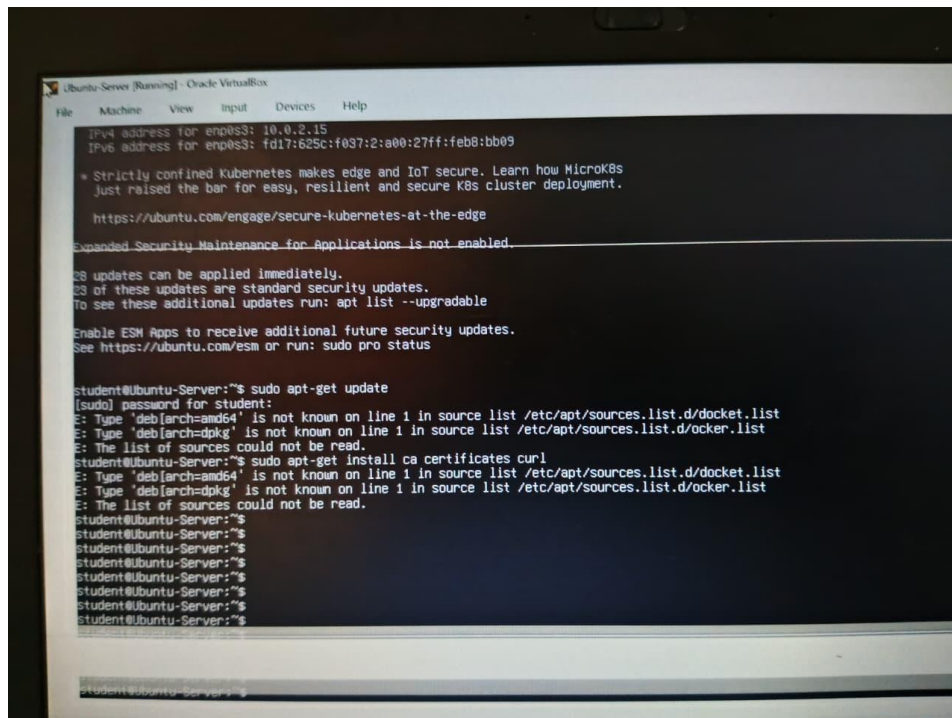
SETUP DOCKER

LANDASAN TEORI

Setup Docker merupakan proses instalasi dan konfigurasi Docker agar dapat digunakan untuk menjalankan aplikasi berbasis container. Docker adalah platform containerization yang memungkinkan aplikasi dijalankan dalam lingkungan terisolasi tanpa harus bergantung pada konfigurasi sistem utama, sehingga lebih ringan dan efisien dibandingkan virtual machine.

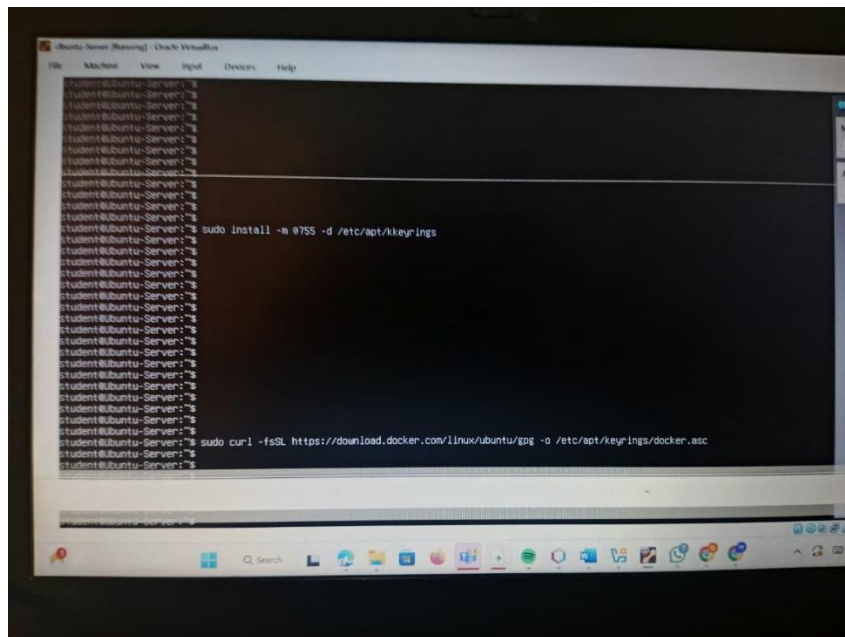
Dalam proses setup, Docker biasanya diinstal pada sistem operasi seperti Ubuntu Server dengan menambahkan repository resmi, mengatur keamanan melalui GPG key, serta melakukan instalasi paket Docker Engine. Setelah instalasi selesai, dilakukan pengujian untuk memastikan Docker dapat berjalan dengan baik.

Dengan setup Docker yang benar, proses deployment aplikasi menjadi lebih cepat, konsisten, dan mudah dikelola. Hal ini sangat penting dalam lingkungan cloud computing karena mendukung skalabilitas serta mempermudah pengembangan dan distribusi aplikasi.

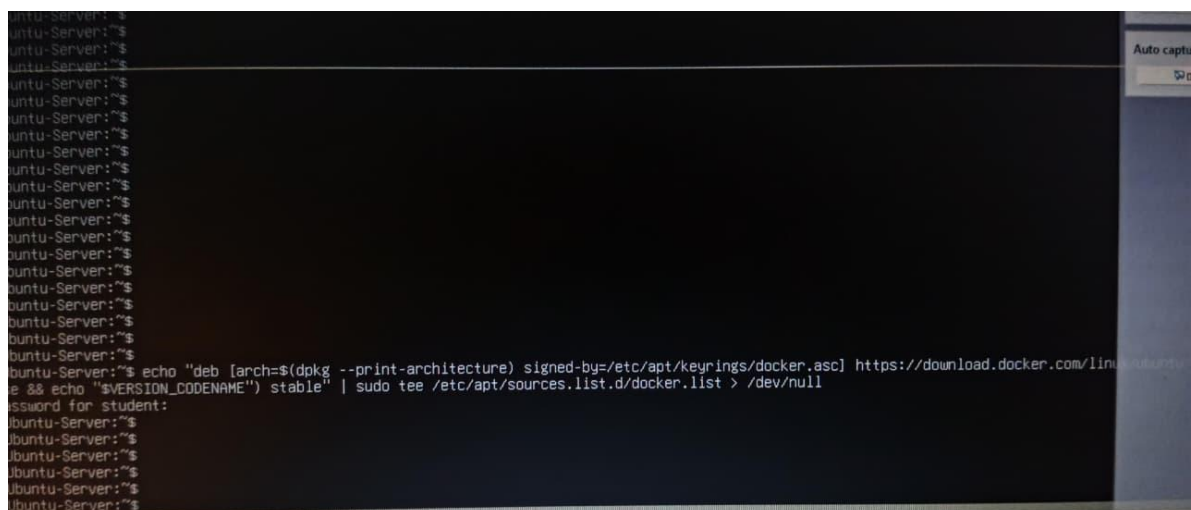


```
student@Ubuntu-Server:~$ sudo apt-get update
[sudo] password for student:
E: Type 'deb[arch=amd64]' is not known on line 1 in source list /etc/apt/sources.list.d/docker.list
E: Type 'deb[arch=dpkg]' is not known on line 1 in source list /etc/apt/sources.list.d/docker.list
E: The list of sources could not be read.
student@Ubuntu-Server:~$ sudo apt-get install ca-certificates curl
E: Type 'deb[arch=amd64]' is not known on line 1 in source list /etc/apt/sources.list.d/docker.list
E: Type 'deb[arch=dpkg]' is not known on line 1 in source list /etc/apt/sources.list.d/docker.list
E: The list of sources could not be read.
student@Ubuntu-Server:~$
student@Ubuntu-Server:~$
student@Ubuntu-Server:~$
student@Ubuntu-Server:~$
student@Ubuntu-Server:~$
student@Ubuntu-Server:~$
student@Ubuntu-Server:~$
student@Ubuntu-Server:~$
```

Masalahnya ada di penulisan **syntax repo Docker yang salah** (contoh: `deb[arch=amd64` tidak terbaca). Akibatnya, sistem tidak bisa membaca source list, jadi install package (termasuk Docker) gagal.



langkah ini digunakan untuk menambahkan keamanan repository Docker agar sistem bisa mengenali dan mempercayai paket yang akan diinstal. Jadi ini tahap awal sebelum install Docker, dan sudah mengarah ke proses yang benar.



Pada tahap ini dilakukan penambahan repository resmi Docker pada sistem Ubuntu Server dengan menggunakan perintah echo yang diarahkan ke file docker.list. Repository ini berfungsi sebagai sumber paket instalasi Docker agar dapat dikenali oleh sistem APT. Dengan menambahkan

repository tersebut, sistem dapat mengunduh dan menginstal Docker dari sumber resmi sehingga proses instalasi menjadi lebih aman dan terstruktur.

```
student@Ubuntu-Server:~$  
student@Ubuntu-Server:~$  
student@Ubuntu-Server:~$  
student@Ubuntu-Server:~$  
student@Ubuntu-Server:~$  
student@Ubuntu-Server:~$  
student@Ubuntu-Server:~$  
student@Ubuntu-Server:~$  
student@Ubuntu-Server:~$  
student@Ubuntu-Server:~$  
student@Ubuntu-Server:~$  
student@Ubuntu-Server:~$  
student@Ubuntu-Server:~$  
student@Ubuntu-Server:~$  
student@Ubuntu-Server:~$  
student@Ubuntu-Server:~$ docker --version  
Docker version 29.4.1, build 055a478  
student@Ubuntu-Server:~$
```

Perintah `docker --version` pada Docker berhasil dijalankan dan menampilkan versi Docker yang terinstal. Hal ini menunjukkan bahwa Docker telah terpasang dengan benar dan siap digunakan pada sistem.

```
student@Ubuntu-Server:~$  
student@Ubuntu-Server:~$ sudo docker run hello-world  
Unable to find image 'hello-world:latest' locally  
  
latest: Pulling from library/hello-world  
4f55086f7dd0: Pull complete  
d5e71e642bf5: Download complete  
Digest: sha256:f9078146db2e05e794366b1bfe584a14ea6317f44027d10ef7dad65279026885  
Status: Downloaded newer image for hello-world:latest  
  
Hello from Docker!  
This message shows that your installation appears to be working correctly.  
  
To generate this message, Docker took the following steps:  
1. The Docker client contacted the Docker daemon.  
2. The Docker daemon pulled the "hello-world" image from the Docker Hub.  
 (amd64)
```

Perintah `sudo docker run hello-world` pada Docker berhasil dijalankan. Sistem dapat mengunduh image dari Docker Hub dan menampilkan pesan “Hello from Docker”, yang menandakan bahwa instalasi Docker berfungsi dengan baik dan siap digunakan.

PERTEMUAN 8

POWER OFF VIRTUAL MACHINE

LANDASAN TEORI

Virtual Machine (VM) adalah teknologi virtualisasi yang memungkinkan satu komputer fisik menjalankan beberapa sistem operasi secara bersamaan. Teknologi ini dijalankan menggunakan software virtualisasi seperti Oracle VirtualBox, yang berfungsi sebagai hypervisor untuk mengelola VM. Power Off Virtual Machine merupakan proses mematikan VM secara langsung, mirip dengan mematikan komputer secara paksa. Pada opsi ini, seluruh proses yang sedang berjalan akan dihentikan tanpa melalui prosedur shutdown sistem operasi di dalam VM. Oleh karena itu, metode ini biasanya digunakan ketika sistem tidak merespons atau dalam kondisi darurat.

Meskipun cepat, penggunaan power off memiliki risiko seperti kehilangan data yang belum tersimpan atau kerusakan sistem file. Oleh karena itu, dalam kondisi normal disarankan menggunakan metode shutdown biasa agar sistem dapat berhenti dengan aman. Dengan demikian, Power Off Virtual Machine merupakan salah satu fitur penting dalam manajemen VM yang harus digunakan secara bijak sesuai kebutuhan.

```
lxion23@lxion:~$ mkdir -p /home/master_app/app mkdir -p /home/master_app/nginx
lxion23@lxion:~$ ls -l /home/master_app
total 12
drwxr-xr-x 2 lxion23 lxion23 4096 Apr 27 17:57 app
-rw-r--r-- 1 root    root    867 Apr 27 16:46 docker-compose.yml
drwxr-xr-x 2 root    root    4096 Apr 27 16:56 nginx
lxion23@lxion:~$ cat docker-compose.yml
cat: docker-compose.yml: No such file or directory
lxion23@lxion:~$ cd /home/master_app
lxion23@lxion:/home/master_app$ cat docker-compose.yml
services:
  mariadb:
    image: mariadb:10.11
    container_name: wp_mariadb
    restart: unless-stopped
    environment:
      MYSQL_ROOT_PASSWORD: rootpassword123
      MYSQL_DATABASE: wordpress
      MYSQL_USER: wpuser
      MYSQL_PASSWORD: wppassword123
    volumes:
      - mariadb_data:/var/lib/mysql
    networks:
      - app-network

  php:
    image: php:8.2-fpm
    container_name: wp_php
    restart: unless-stopped
    volumes:
      - ./app:/var/www/html
    networks:
      - app-network
    depends_on:
      - mariadb

  nginx:
    image: nginx:latest
    container_name: wp_nginx
    restart: unless-stopped
    ports:
      - "80:80"
    volumes:
      - ./app:/var/www/html
      - ./nginx/default.conf:/etc/nginx/conf.d/default.conf
    networks:
      - app-network
    depends_on:
      - php

volumes:
  mariadb_data:

networks:
  app-network:
lxion23@lxion:/home/master_app$ |
```


Pada tahap ini dilakukan persiapan folder dan file untuk proyek multi-container menggunakan Docker Compose. Folder app dan nginx berhasil dibuat di direktori /home/master_app. Selain itu, file docker-compose.yml telah dibuat dan berisi konfigurasi layanan seperti MariaDB, PHP, dan Nginx yang saling terhubung dalam satu network. Tahap ini menunjukkan bahwa struktur awal proyek telah siap digunakan.

```
lxion23@lxion:/home/master_app$ cat nginx/default.conf
server {
    listen 80;
    server_name localhost;

    root /var/www/html;
    index index.php index.html;

    location / {
        try_files $uri $uri/ /index.php?$args;
    }

    location ~ \.php$ {
        include fastcgi_params;
        fastcgi_pass wp_php:9000;
        fastcgi_index index.php;
        fastcgi_param SCRIPT_FILENAME $document_root$fastcgi_script_name;
    }

    location ~ /\.ht {
        deny all;
    }
}
lxion23@lxion:/home/master_app$ |
```

File konfigurasi default.conf pada Nginx berhasil dibuat. Konfigurasi ini mengatur server untuk berjalan di port 80, menentukan root direktori, serta menghubungkan pemrosesan file PHP ke service PHP (FastCGI). Hal ini menunjukkan bahwa web server telah dikonfigurasi untuk mendukung aplikasi berbasis PHP.

```
Swap usage:          0%
Processes:           112
Users logged in:     0
IPv4 address for enp0s3: 10.0.2.15
IPv6 address for enp0s3: fd17:625c:f037:2:a00:27ff:fe4f:1b88

Expanded Security Maintenance for Applications is not enabled.

59 updates can be applied immediately.
To see these additional updates run: apt list --upgradable

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status

The programs included with the Ubuntu system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

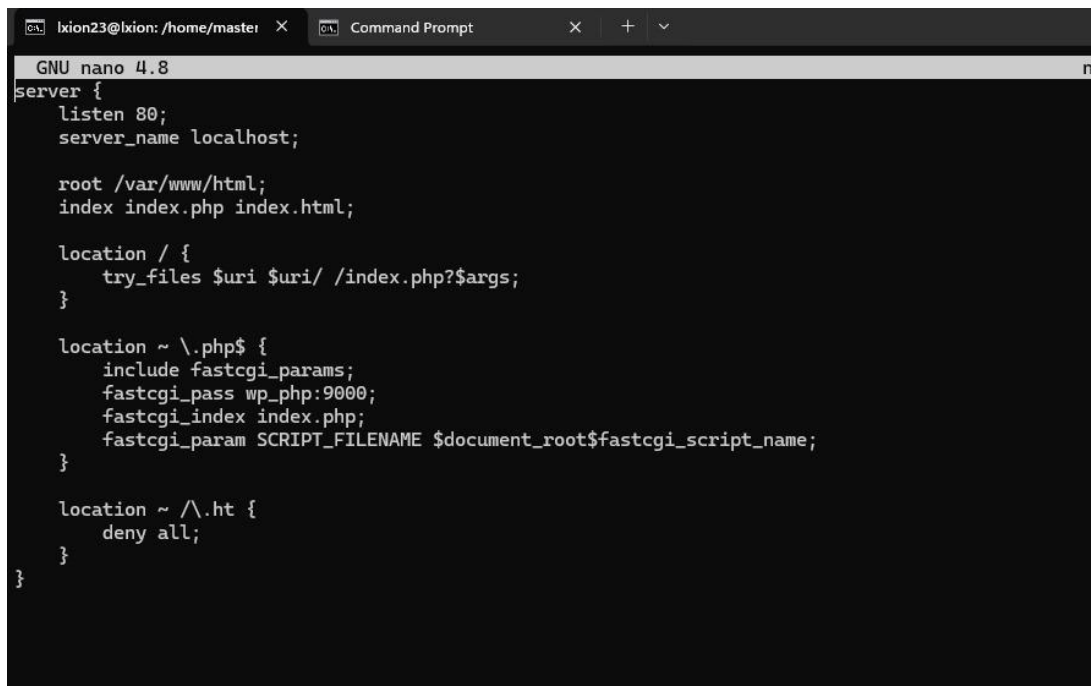
Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by
applicable law.

To run a command as administrator (user "root"), use "sudo <command>".
See "man sudo_root" for details.

ryan@Ubuntu-serverr:~$ sudo apt update -y
[sudo] password for ryan:
Hit:1 http://security.ubuntu.com/ubuntu jammy-security InRelease
Hit:2 http://id.archive.ubuntu.com/ubuntu jammy InRelease
Hit:3 http://id.archive.ubuntu.com/ubuntu jammy-updates InRelease
Hit:4 http://id.archive.ubuntu.com/ubuntu jammy-backports InRelease
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
58 packages can be upgraded. Run 'apt list --upgradable' to see them.
ryan@Ubuntu-serverr:~$ sudo apt install nginx -y
```

Pembuatan file docker-compose.yml menggunakan editor nano pada Ubuntu Server berhasil dilakukan. File ini digunakan untuk mendefinisikan dan mengelola layanan container dalam Docker

Compose, sehingga memudahkan konfigurasi dan menjalankan beberapa container secara bersamaan.



```
GNU nano 4.8
server {
    listen 80;
    server_name localhost;

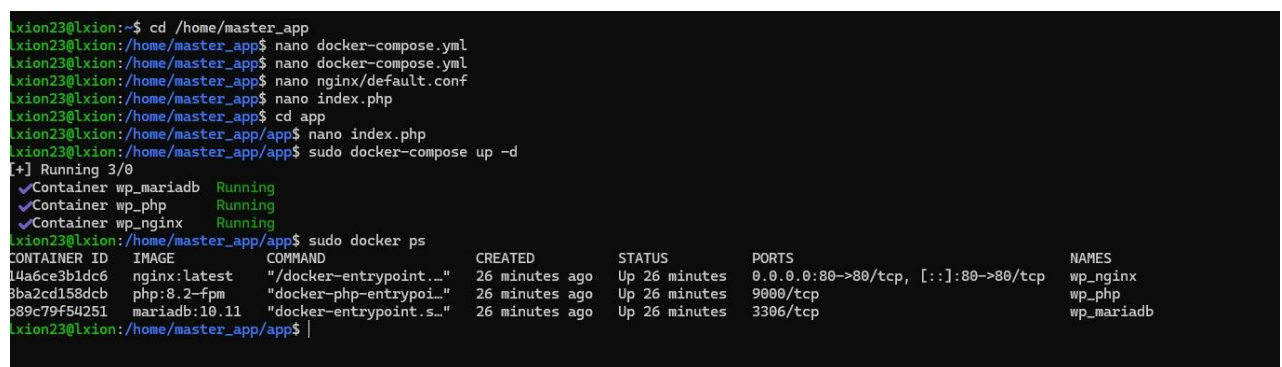
    root /var/www/html;
    index index.php index.html;

    location / {
        try_files $uri $uri/ /index.php?$args;
    }

    location ~ \.php$ {
        include fastcgi_params;
        fastcgi_pass wp_php:9000;
        fastcgi_index index.php;
        fastcgi_param SCRIPT_FILENAME $document_root$fastcgi_script_name;
    }

    location ~ /\.ht {
        deny all;
    }
}
```

Pembuatan file default.conf menggunakan editor nano pada Nginx berhasil dilakukan. File ini berisi konfigurasi server untuk menangani request HTTP dan menghubungkan pemrosesan PHP melalui FastCGI. Hal ini menunjukkan bahwa konfigurasi web server telah disiapkan dengan baik.



```
lxion23@lxion:~$ cd /home/master_app
lxion23@lxion:/home/master_app$ nano docker-compose.yml
lxion23@lxion:/home/master_app$ nano docker-compose.yml
lxion23@lxion:/home/master_app$ nano nginx/default.conf
lxion23@lxion:/home/master_app$ nano index.php
lxion23@lxion:/home/master_app$ cd app
lxion23@lxion:/home/master_app/app$ nano index.php
lxion23@lxion:/home/master_app/app$ sudo docker-compose up -d
[+] Running 3/0
  ✓ Container wp_mariadb  Running
  ✓ Container wp_php      Running
  ✓ Container wp_nginx    Running
lxion23@lxion:/home/master_app/app$ sudo docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS                               NAMES
l4a6ce3b1dc6   nginx:latest   "/docker-entrypoint..." 26 minutes ago Up 26 minutes 0.0.0.0:80->80/tcp, [::]:80->80/tcp wp_nginx
8ba2cd158dcb   php:8.2-fpm    "docker-php-entrypoi..." 26 minutes ago Up 26 minutes 9000/tcp                               wp_php
p89c79f54251   mariadb:10.11  "docker-entrypoint.s..." 26 minutes ago Up 26 minutes 3306/tcp                               wp_mariadb
lxion23@lxion:/home/master_app/app$
```

Perintah docker-compose up -d pada Docker Compose berhasil dijalankan. Semua container (MariaDB, PHP, dan Nginx) berjalan dengan status running, yang menunjukkan bahwa konfigurasi multi-container telah berhasil dijalankan dan sistem siap digunakan.

```
GNU nano 4.8 index.php
<!DOCTYPE html>
<html lang="id">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Profil Dini Yusriah</title>
  <style>
    * {
      margin: 0;
      padding: 0;
      box-sizing: border-box;
    }

    body {
      font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
      min-height: 100vh;
      background: linear-gradient(135deg, #ffe4f1, #f7d9ff, #fff4fb);
      display: flex;
      justify-content: center;
      align-items: center;
      padding: 30px 15px;
      overflow-x: hidden;
    }

    .container {
      width: 100%;
      max-width: 900px;
      position: relative;
    }

    .bg-shape-1,
    .bg-shape-2 {
      position: absolute;
      border-radius: 50%;
      filter: blur(10px);
      z-index: 0;
    }

    .bg-shape-1 {
      width: 180px;
      height: 180px;
      background: rgba(255, 182, 193, 0.35);
    }
```

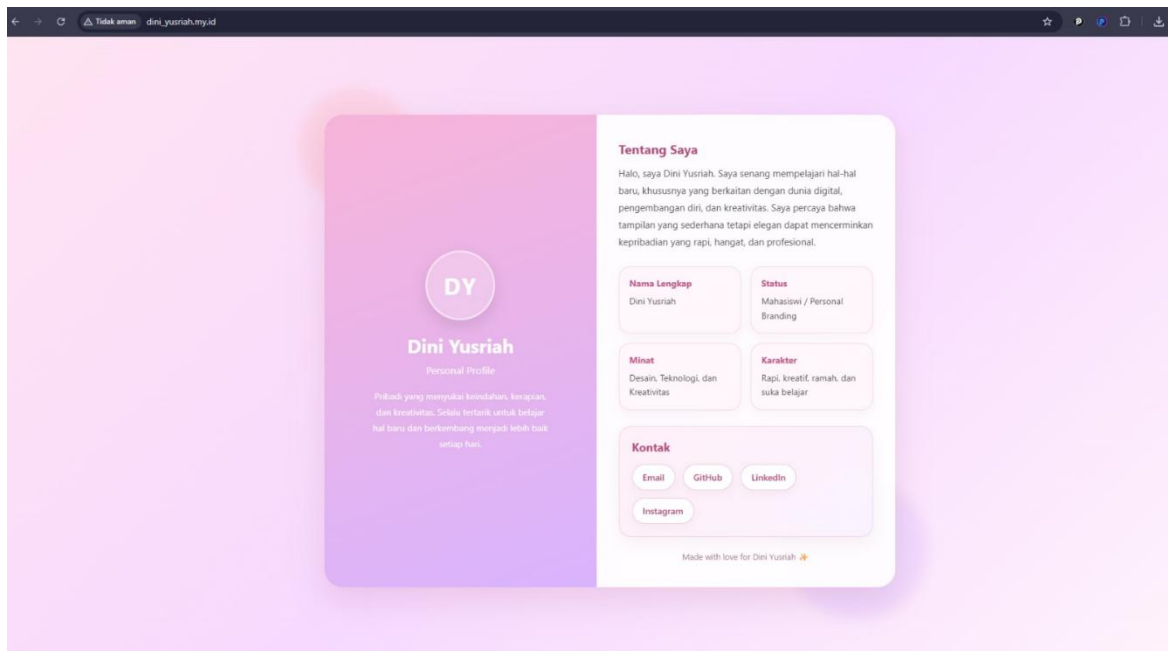
File index.php berhasil dibuat menggunakan editor nano dan berisi struktur dasar HTML serta CSS untuk tampilan halaman web. File ini berfungsi sebagai halaman utama yang akan ditampilkan oleh Nginx, sehingga menunjukkan bahwa konten web telah berhasil disiapkan dan siap diakses melalui browser.

```
GNU nano 4.8
127.0.0.1 localhost
127.0.0.1 lxion

# The following lines are desirable for IPv6 capable hosts
::1 ip6-localhost ip6-loopback
fe00::0 ip6-localnet
ff00::0 ip6-mcastprefix
ff02::1 ip6-allnodes
ff02::2 ip6-allrouters

192.168.1.22 Dini_Yusriah.my.id
```

Konfigurasi domain dilakukan dengan menambahkan mapping IP dan nama domain pada file hosts di Ubuntu Server. Domain Dini_Yusriah.my.id berhasil diarahkan ke alamat IP lokal, sehingga memungkinkan akses website melalui nama domain tersebut di jaringan lokal.



Konfigurasi domain dilakukan dengan menambahkan mapping IP dan nama domain pada file hosts di Ubuntu Server. Domain Dini_Yusriah.my.id berhasil diarahkan ke alamat IP lokal, sehingga memungkinkan akses website melalui nama domain tersebut di jaringan lokal.

KESIMPULAN

Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa penggunaan teknologi seperti Nginx, Virtual Machine, dan container dengan Docker mempermudah proses deployment dan pengelolaan aplikasi berbasis cloud. Praktikum ini menunjukkan bahwa aplikasi dapat dijalankan baik di lingkungan cloud maupun lokal dengan konfigurasi yang tepat.

Dengan demikian, praktikum ini memberikan pemahaman dasar yang baik mengenai konsep Cloud Computing, mulai dari instalasi, konfigurasi, hingga implementasi layanan, sehingga dapat menjadi bekal dalam pengembangan sistem yang lebih kompleks di masa depan.